SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : **vadapalli s.k.pavan.p.ramacharyulu**

Reg.No : **11249A393**

Class : **II B.E(CSE)**

Subject Code :

Subject Name :

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by

Mr/Ms._____Vadapalli s.k.pavan.p.ramacharyulu_____
with Reg. No 11249A393 done by of II-B.E.(CSE) in dst
for Lab during the academic year 2025 -2026.

Station : ENATHUR

Date :

Staf-in-charge **Head of the Department**

Submitted for the Practical examination held on_____

Examine-1 **Examine -2**

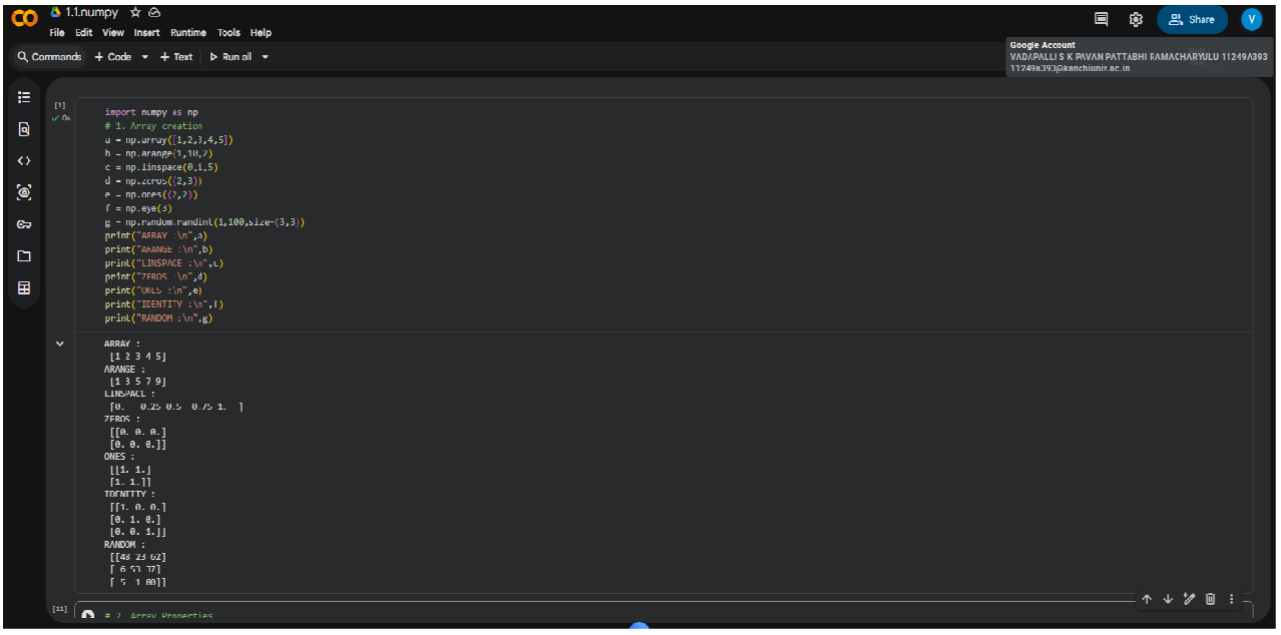
INDEX

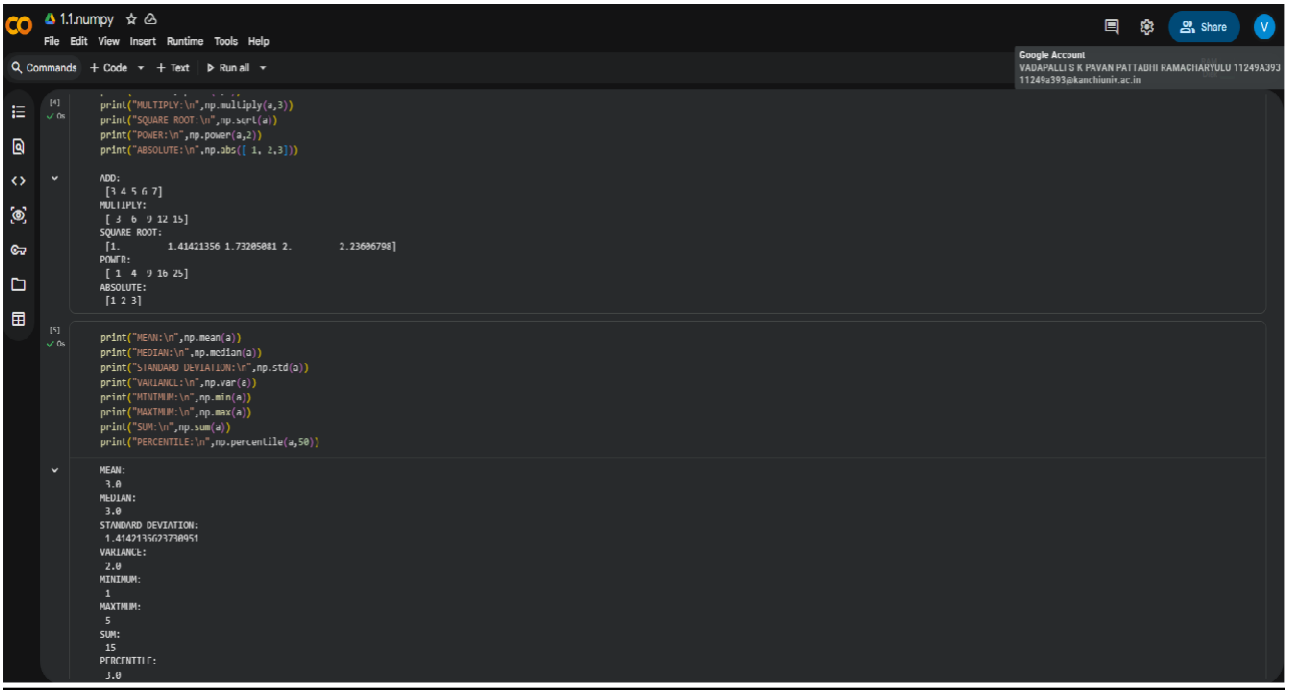
SI.NO	Program Name	Date	Page .no	Signature
01 (1.1)	Introduction to Python Libraries for Data Science			
1.2	Pandas			
1.3	Matplotlib			
2	Perform Data Explorationand Pre-processing in Python			
3	Implement Regularized Linear Regression			
4	Implement Naive Bayes Classifier			
5	Implement Regularized Logistic Regression			
6	Build Modules Using Different Ensembling Techniques			
7	Build Models Using Decision Trees			
8	Build the Models Using Different Kernel Functions			
9	Implement K-Nearest Neighbours (KNN) Classification			
10	K-Mean Clusteringwith PCAand Elbow Method			

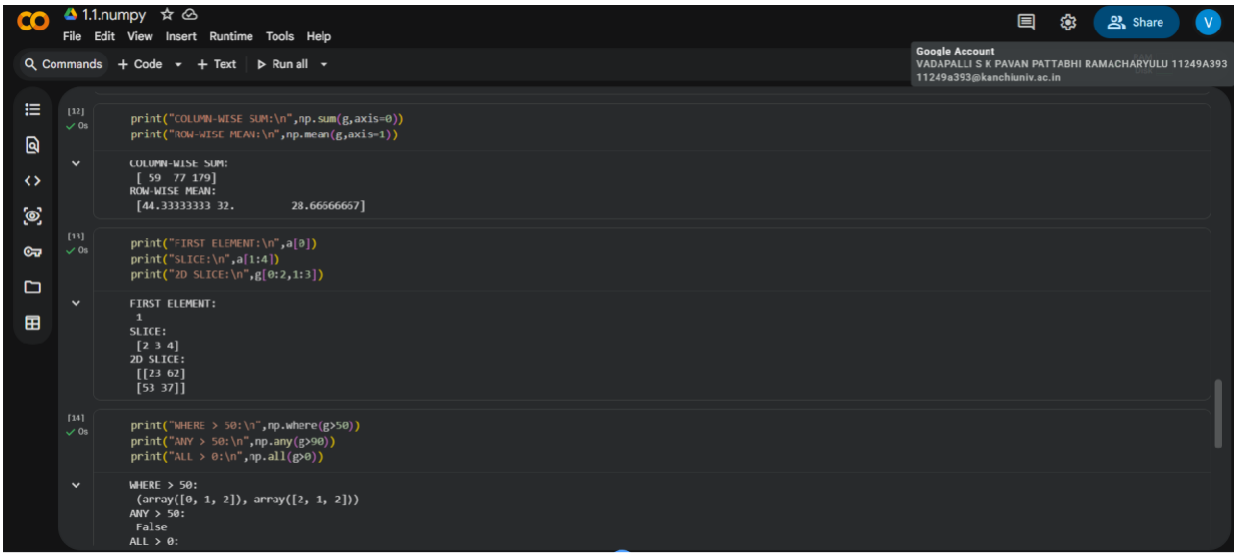
INTRODUCTION TO PYTHON LIBRARIES FOR DATASCIENCE

EXP.NO:-1.1

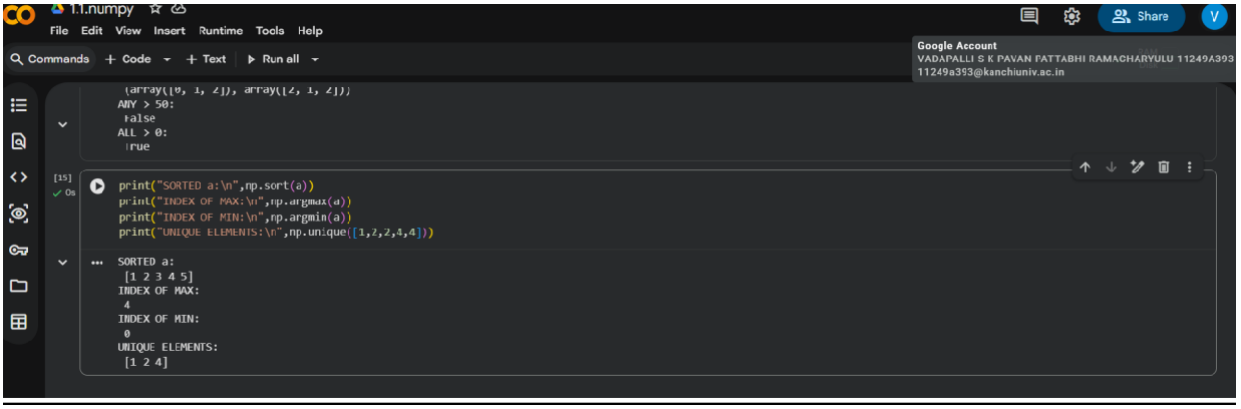
Aim: To understand the basic Python libraries used in data science, such as NumPy, Pandas, Matplotlib, and Scikit-learn.







```
[12]
print("COLUMN-WISE SUM:\n",np.sum(g,axis=0))
print("ROW-WISE MEAN:\n",np.mean(g,axis=1))

COLUMN-WISE SUM:
 [ 59  77 179]
ROW-WISE MEAN:
 [44.33333333 32.         28.66666667]

[11]
print("FIRST ELEMENT:\n",a[0])
print("SLICE:\n",a[1:4])
print("2D SLICE:\n",g[0:2,1:3])

FIRST ELEMENT:
 1
SLICE:
 [2 3 4]
2D SLICE:
 [[23 62]
 [53 37]]

[14]
print("WHERE > 50:\n",np.where(g>50))
print("ANY > 50:\n",np.any(g>90))
print("ALL > 0:\n",np.all(g>0))

WHERE > 50:
 (array([0, 1, 2]), array([2, 1, 2]))
ANY > 50:
 False
ALL > 0:
```



```
(array([0, 1, 2]), array([2, 1, 2]))
ANY > 50:
 False
ALL > 0:
 True

[15]
print("SORTED a:\n",np.sort(a))
print("INDEX OF MAX:\n",np.argmax(a))
print("INDEX OF MIN:\n",np.argmin(a))
print("UNIQUE ELEMENTS:\n",np.unique([1,2,2,4,4]))

SORTED a:
 [1 2 3 4 5]
INDEX OF MAX:
 4
INDEX OF MIN:
 0
UNIQUE ELEMENTS:
 [1 2 4]
```

Result: Understood the role of Python libraries in handling, analyzing, and visualizing data. Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn are commonly used for these tasks.

EXP.NO:-1.2 Pandas

Aim: To learn how to use Pandas for data loading, manipulation, and analysis.

PANDAS

A)creating dataframes

```
import pandas as pd
data ={
    'Name' :['Alice','Bob','charlie'],
    'Age' : [25,30,35],
    'City' : ['Newyork','Paris','London']
}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```

```
      Name  Age     City
0    Alice   25  Newyork
1      Bob   30    Paris
2  charlie   35   London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
       Age
count  3.0
mean  30.0
std    5.0
min   25.0
25%   27.5
50%   30.0
75%   32.5
max   35.0
```

```
max    35.0
```

B)SELECTING AND FILTERING

```
ages = df ['Age']
above_25 = df[df['Age']>25]
row_0 =df.iloc[0]
specific_val =df.loc[0,'City']
print(ages)
print(above_25)
print(row_0)
print(specific_val)
```

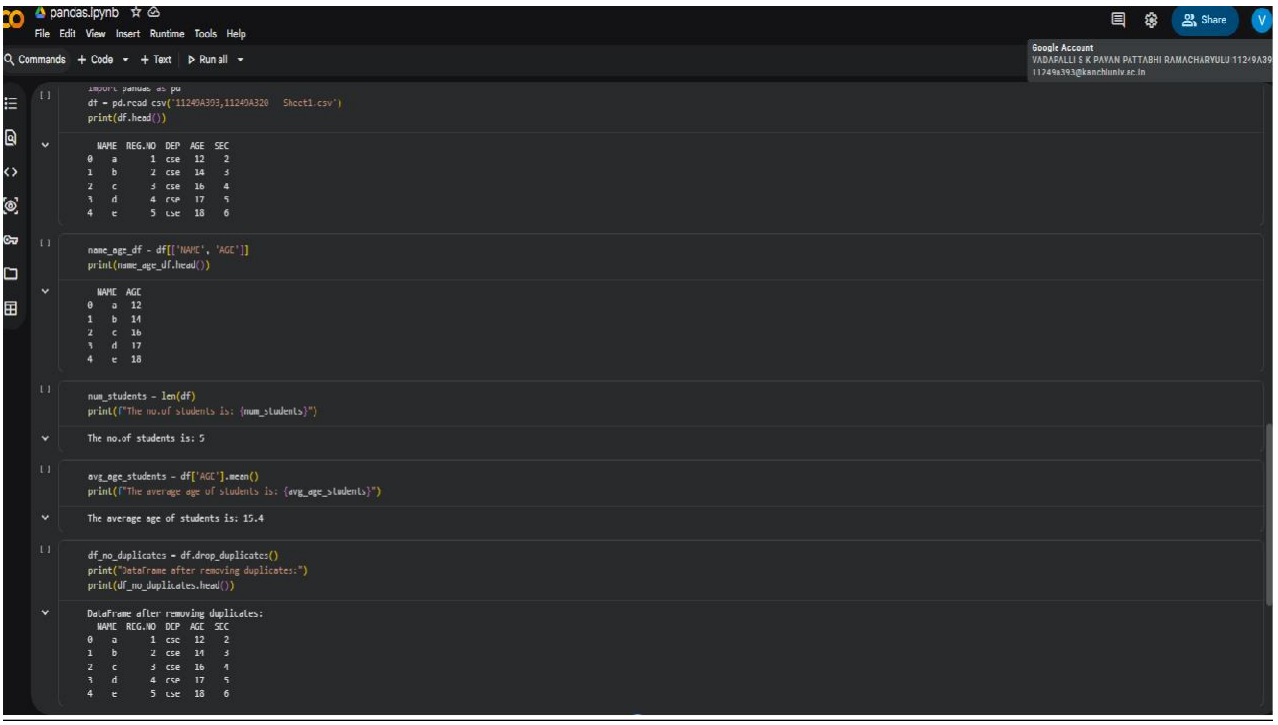
```
0    25
1    30
2    35
Name: Age, dtype: int64
      Name  Age    City
1      Bob   30   Paris
2  charlie   35  London
Name       Alice
Age           25
City     Newyork
Name: 0, dtype: object
Newyork
```

Show hidden output

C)Data Cleaning(Handling Missimg Values)

```
print(df.isnull().sum())
df_cleane=df.dropna()
df_filled =df.fillna(0)
print(df_cleane)
print(df_filled)
```

```
Name    0
Age     0
City    0
dtype: int64
      Name  Age     City
0    Alice   25  Newyork
1      Bob   30    Paris
2  charlie   35   London
      Name  Age     City
0    Alice   25  Newyork
1      Bob   30    Paris
2  charlie   35   London
```

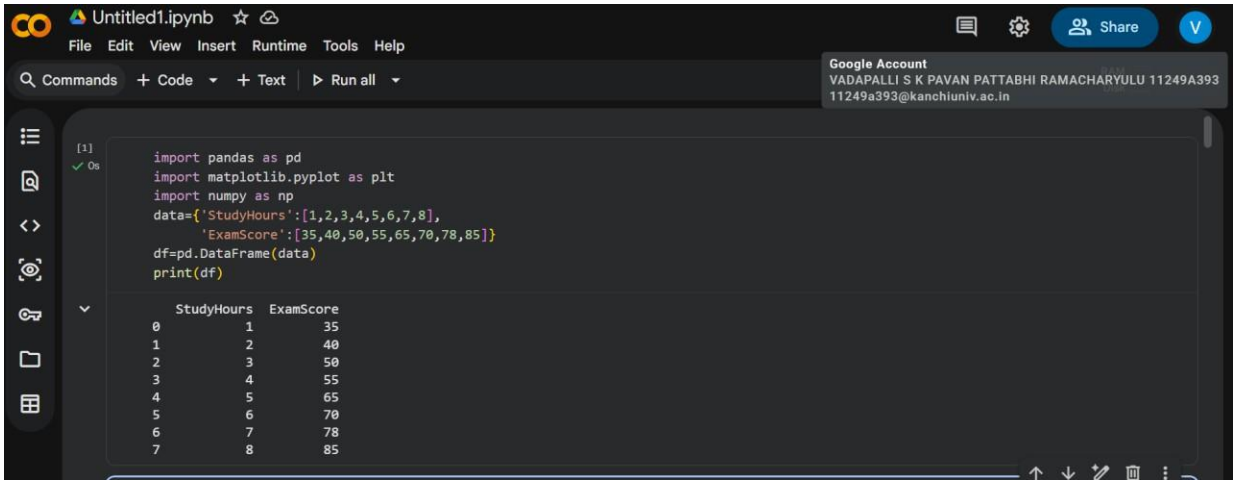


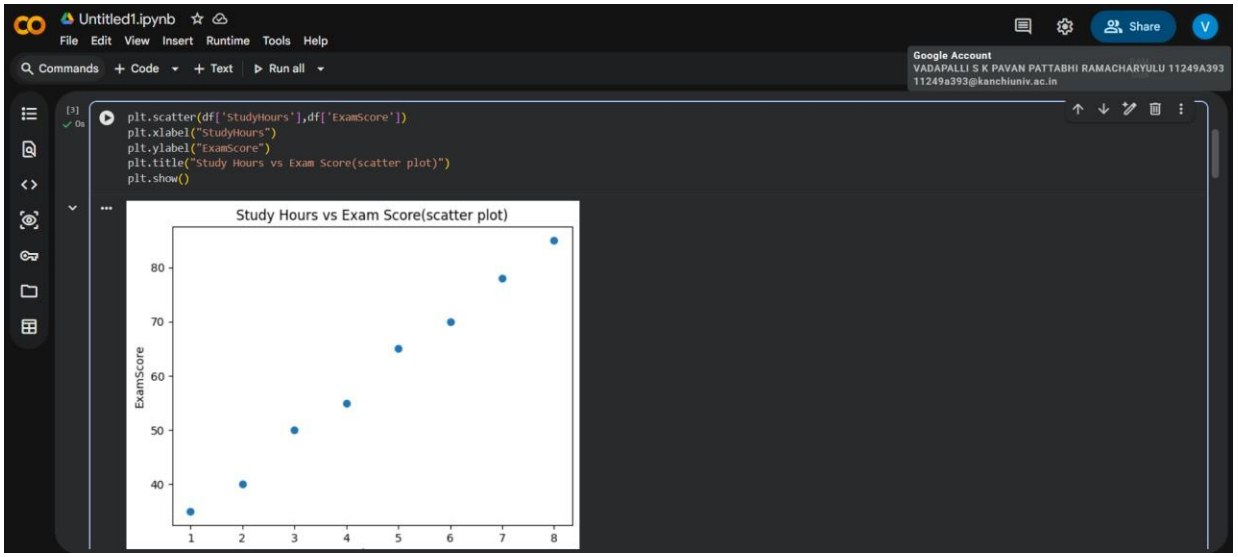
```
pandas.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 112149A39
112149a393@kanchiuniv.ac.in

import pandas as pd
df = pd.read_csv('11240A393,11240A320  Sheet1.csv')
print(df.head())

  NAME  REG.NO  DEP  AGE  SEC
0    a       1  cse   12    2
1    b       2  cse   14    3
2    c       3  cse   16    4
3    d       4  cse   17    5
4    e       5  cse   18    6

name_age_df = df[['NAME', 'AGE']]
print(name_age_df.head())

  NAME  AGE
0    a   12
1    b   14
2    c   16
3    d   17
4    e   18

num_students = len(df)
print(f"The no.of students is: {num_students}")

The no.of students is: 5

avg_age_students = df['AGE'].mean()
print(f"The average age of students is: {avg_age_students}")

The average age of students is: 15.4

df_no_duplicates = df.drop_duplicates()
print("DataFrame after removing duplicates:")
print(df_no_duplicates.head())

DataFrame after removing duplicates:
  NAME  REG.NO  DEP  AGE  SEC
0    a       1  cse   12    2
1    b       2  cse   14    3
2    c       3  cse   16    4
3    d       4  cse   17    5
4    e       5  cse   18    6
```

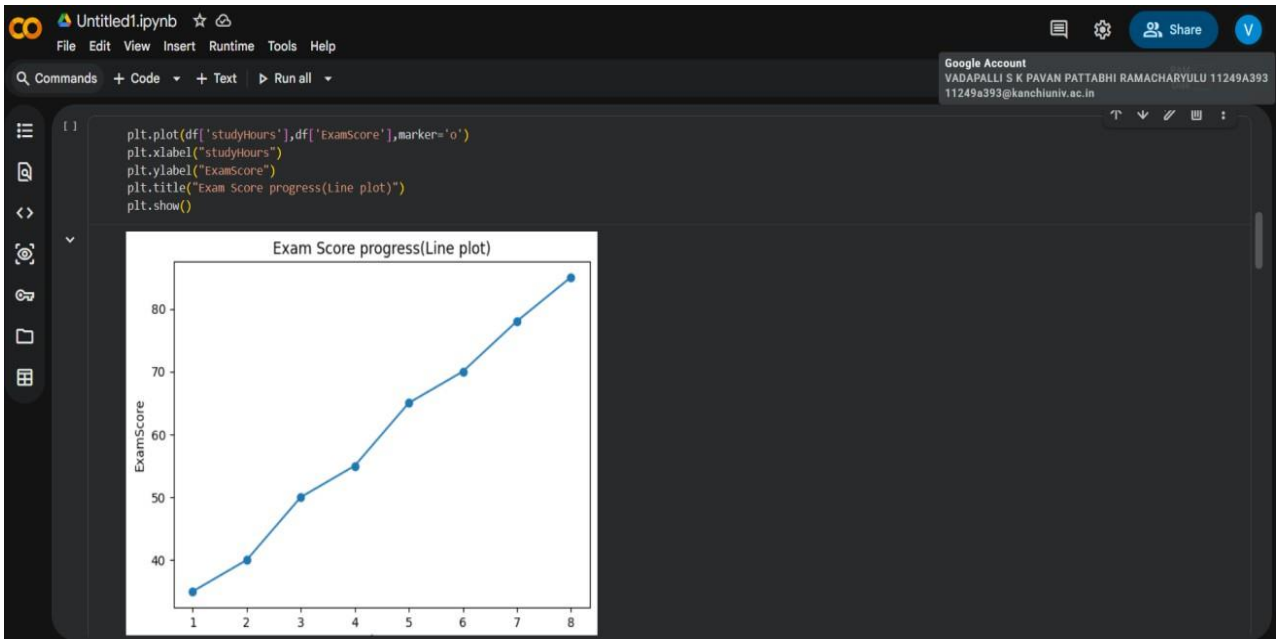
Result: Successfully performed data manipulation tasks such as creating, modifying, cleaning, filtering, and summarizing datasets. Pandas is widely used for preparing data for analysis and machine learning.

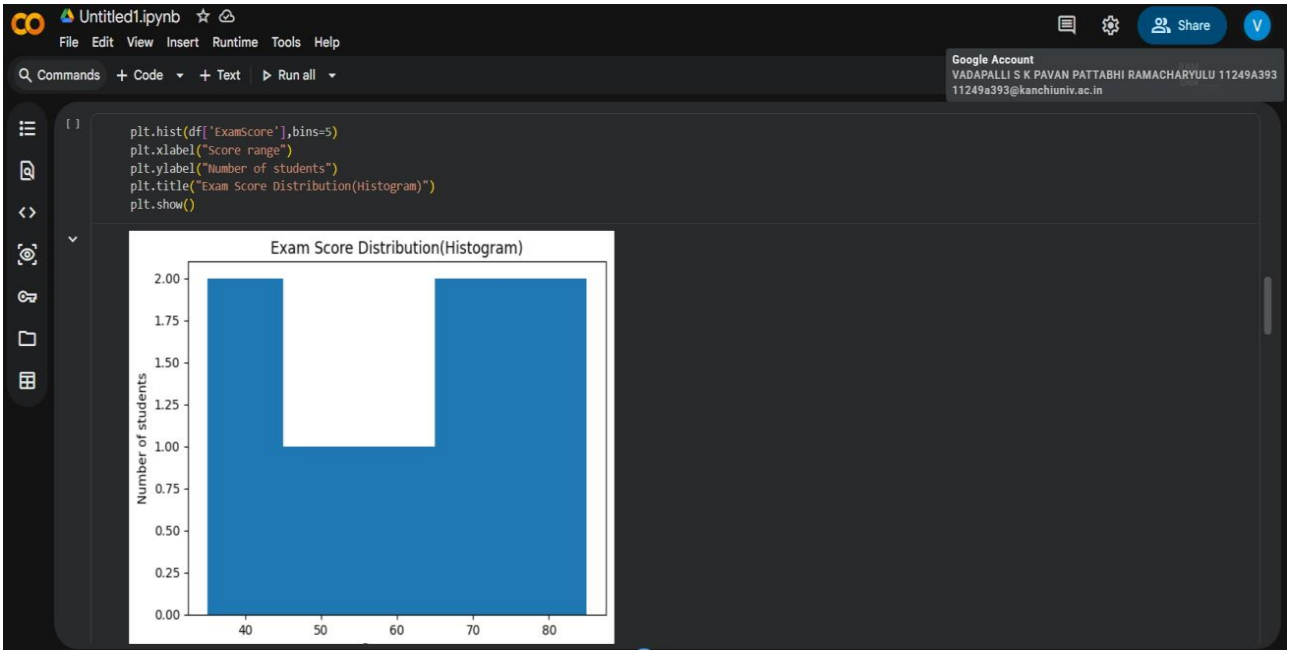
EXP.NO:-1.3 Matplotlib

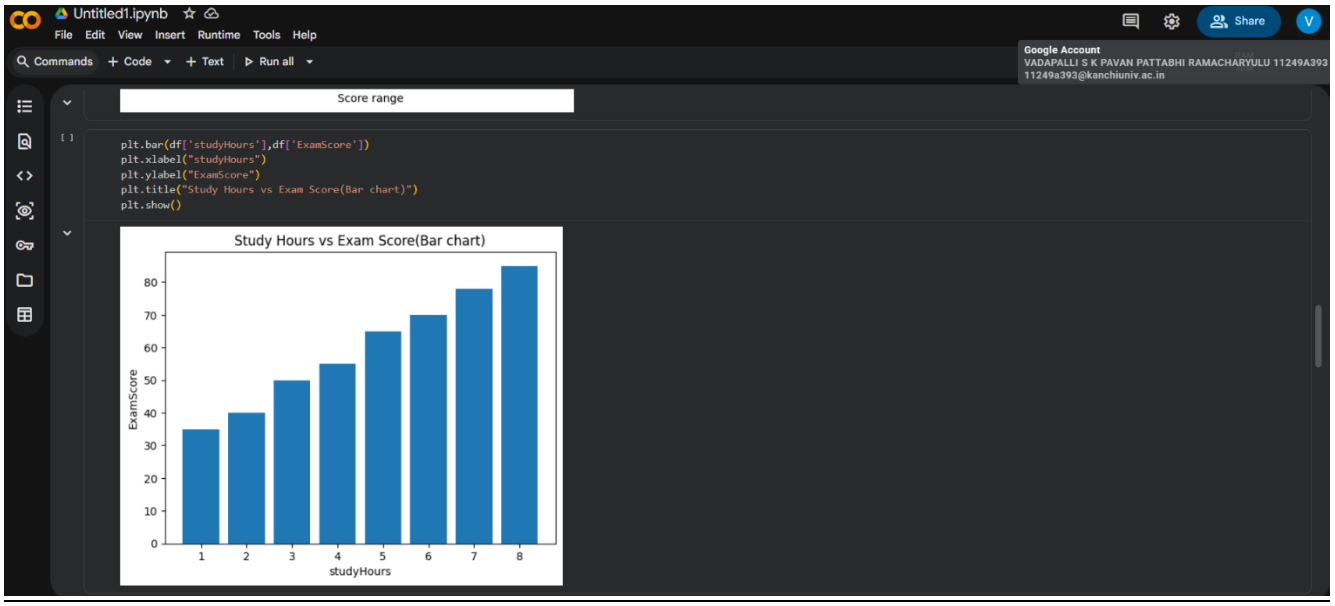
Aim: To learn how to create graphical visualizations using Matplotlib.






Untitled1.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in
plt.plot(df['studyHours'],df['ExamScore'],marker='o')
plt.xlabel("studyHours")
plt.ylabel("ExamScore")
plt.title("Exam Score progress(Line plot)")
plt.show()
Exam Score progress(Line plot)
ExamScore
80
70
60
50
40
1
2
3
4
5
6
7
8


Untitled1.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in
plt.hist(df['ExamScore'],bins=5)
plt.xlabel("Score range")
plt.ylabel("Number of students")
plt.title("Exam Score Distribution(Histogram)")
plt.show()
Exam Score Distribution(Histogram)
Number of students
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
40
50
60
70
80



Result: Successfully created plots such as line charts, bar charts, scatter plots, and other basic graphs for data presentation.Matplotlib is commonly used for visualizing x and y points and customizing plots

EXP.NO:-1.4 Scikit-learn

Aim: To understand the use of Scikit-learn for building machine learning models.

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAFALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

STEP 1: IMPORT LIBRARIES

[1]
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

STEP 2: CREATE DATASET

[2]
```
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
```

```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              38
2      S3                40              42
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
```

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
7     S8                65              65
8     S9                70              70
9    S10                75              75
```

STEP 3: FEATURES & TARGET

+ Code + Text

[3]
```
x = df[['Classes_Attended']]
y = df['Internal_Marks']
print(x)
print(y)
```

```
   Classes_Attended
0                30
1                35
2                40
3                45
4                50
5                55
6                60
7                65
8                70
9                75
0    35
1    38
2    42
3    46
4    50
5    55
6    60
7    65
8    70
9    75
Name: Internal_Marks, dtype: int64
```

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

STEP 4: TRAIN-TEST SPLIT

[9]

```
X_train, X_test, y_train, y_test = train_test_split(
x, y, test_size=0.2, random_state=7
)
```

[10]

```
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)

x = df[['Classes_Attended']]
y = df['Internal_Marks']
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=7)

model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

[11]

```
predictions = model.predict(X_test)
```

Untitled2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

[11]

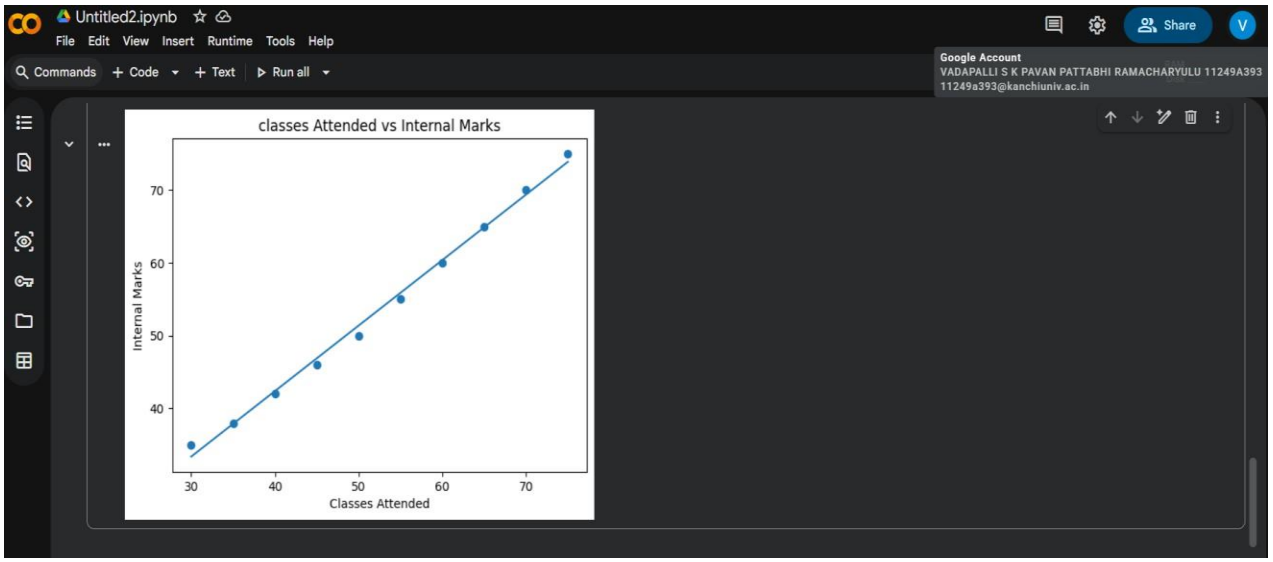
```
predictions = model.predict(X_test)
```

[13]

```
mse = mean_squared_error(y_test, predictions)

print("MSE:", mse)

print("Marks per class:", model.coef_[0])
print("Base Marks:", model.intercept_)
```

```
MSE: 0.578125
Marks per class: 0.9
Base Marks: 6.375
```

OUTPUT:



<u>Result: Learned how to apply Scikit-learn for training, testing, and evaluating machine learning models in data science tasks.</u>

EXP.NO:02

Perform Data Exploration and Pre-processing in Python

Aim: To explore datasets and prepare them for analysis by cleaning and transforming the data.

EXP_2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

```
import numpy as np
import pandas as pd

data={
    "country":['indian ','usa','indian','usa','uk','indian'],
    "age ":[22,25,np.nan,30,28,35],
    "salary":[40000,60000,50000,np.nan,72000,58000],
    "purchased":['no' , 'yes','yes','no','yes','yes']
}
df=pd.DataFrame(data)
print("__original raw data__")
print(df)
print("\n")
```

```
__original raw data__
   country  age    salary purchased
0  indian   22.0  40000.0        no
1     usa   25.0  60000.0       yes
2  indian    NaN  50000.0       yes
3     usa   30.0      NaN        no
4      uk   28.0  72000.0       yes
5  indian   35.0  58000.0       yes
```

EXP_2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

```
4       uk  28.0  72000.0       yes
5  indian  35.0  58000.0       yes
```

```
print("__missing values count__")
print(df.isnull().sum())
print("\n")
```

```
__missing values count__
country      0
age          1
salary       1
purchased    0
dtype: int64
```

EXP_2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

[4]

```python
df_encoded = pd.get_dummies(df,columns=['country'])
df_encoded['purchased']=df_encoded['purchased'].map({'yes':1,'no':0})
print("__data after encoding (all numbers now)")
print(df_encoded)
print("\n")
```

```
__data after encoding (all numbers now)
   age    salary  purchased  country_indian  country_indian   country_uk  \
0  22.0  40000.0          0           False            True       False
1  25.0  60000.0          1           False           False       False
2   NaN  50000.0          1            True           False       False
3  30.0      NaN          0           False           False       False
4  28.0  72000.0          1           False           False        True
5  35.0  58000.0          1            True           False       False

   country_usa
0        False
1         True
2        False
3         True
4        False
5        False
```

EXP_2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

[6]

```python
from sklearn.preprocessing import StandardScaler
x= df_encoded.drop('purchased',axis=1)
y=df_encoded['purchased']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(x)
print("__FINAL PROCESSED DATA (ready for ai)___")
print(pd.DataFrame(X_scaled,columns=x.columns).head())
```

```
__FINAL PROCESSED DATA (ready for ai)___
        age     salary  country_indian  country_indian   country_uk  \
0 -1.355262 -1.501173       -0.707107         2.236068   -0.447214
1 -0.677631  0.375293       -0.707107        -0.447214   -0.447214
2       NaN -0.562940        1.414214        -0.447214   -0.447214
3  0.451754       NaN       -0.707107        -0.447214   -0.447214
4  0.000000  1.501173       -0.707107        -0.447214    2.236068

   country_usa
0    -0.707107
1     1.414214
2    -0.707107
3     1.414214
4    -0.707107
```

Result: Successfully performed data exploration, handled missing values, encoded categorical features, and scaled data.

EXP.NO:03

Implementation regularized linear regression

AIM:- To implement linear regression with regularization techniques to improve model performance.

EXP_3(linear regression)

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

[1]

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets # Import datasets
from sklearn.model_selection import train_test_split # Import train_test_split

diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(diabetes.data,
diabetes.target, test_size=0.2, random_state=42)


lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test,
lr.predict(X_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test,
ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test,
lasso.predict(X_test)):.2f}")
```

Output:-

EXP_3(linear regression)

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

[1]

```
print(f"Ridge Regression MSE: {mean_squared_error(y_test,
ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test,
lasso.predict(X_test)):.2f}")
```

```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

Result: Built a regularized linear regression model and observed reduced overfitting

EXP.NO:04

Implementation navie bayes classified

AIM:- To implement a Naive Bayes classification model for prediction tasks

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}

df = pd.DataFrame(data)

X = df[['StudyHours', 'Attendance']]
y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Predicted Results:", y_pred)
print("Actual Results: ", y_test.values)
```

output:-

```
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Results:", y_pred)
print("Actual Results: ", y_test.values)
print("Accuracy:", accuracy)

Predicted Results: ['Fail' 'Fail']
Actual Results:  ['Fail' 'Fail']
Accuracy: 1.0
```

Result: Successfully trained and tested a Naive Bayes classifier and obtained classification results.

EXP.NO:05

Logistic regression

AIM: To implement logistic regression with regularization for classification.

exp 5(logistic regression)

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

Share

[1]

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


data = load_breast_cancer()
X = data.data
y = data.target


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
```

exp 5(logistic regression)

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

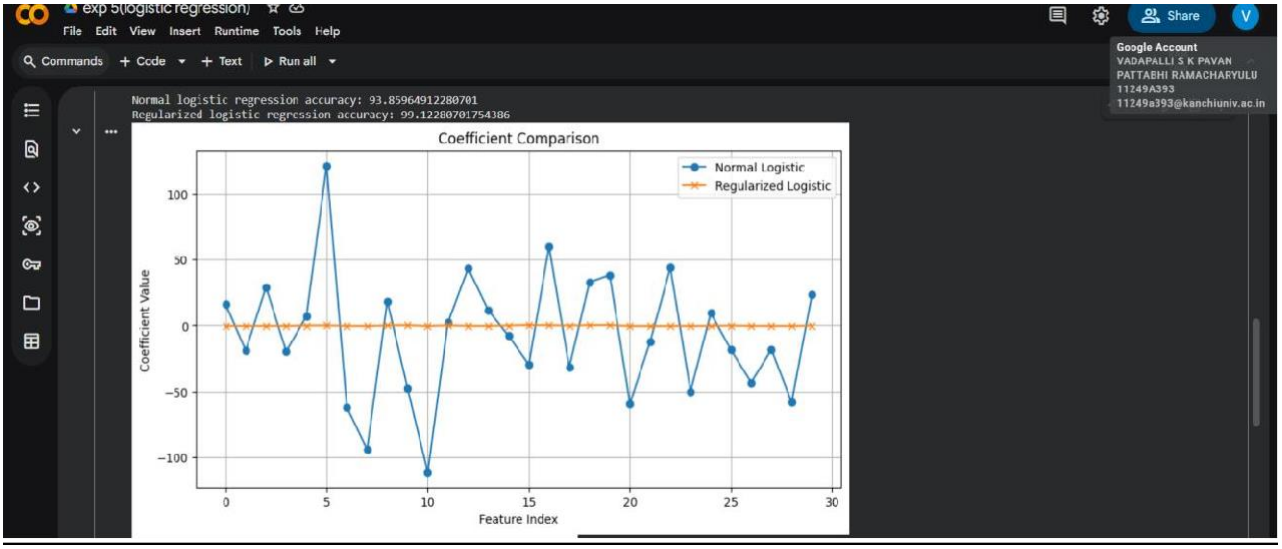
Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

Share

[1]

```python
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)


print("Normal logistic regression accuracy:", normal_acc * 100)
print("Regularized logistic regression accuracy:", reg_acc * 100)


plt.figure(figsize=(10,5))
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic', marker='o')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic', marker='x')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid(True)
plt.show()


plt.figure(figsize=(6,4))
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies, color=['blue', 'orange'])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0,1)
plt.show()
```

output:-



Result: Built a regularized logistic regression model and evaluated its classification accuracy.

EXP.NO:06

Build models using different ensembilng techniques

AIM:- To understand and implement ensemble learning methods.

exp 6

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

6 Build Models using Different Ensembling Techniques

```
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

output:-

exp 6

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

```
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

Result: Successfully built models using ensemble techniques such as bagging, boosting, or random forest

EXP.NO:07

Build models using decision trees

Aim: To construct decision tree models for classification or regression.

exp 7

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

7.Build Models Using Decision Tree

[1]

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

Output:-

exp 7

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Google Account
VADAPALLI S K PAVAN PATTABHI RAMACHARYULU 11249A393
11249a393@kanchiuniv.ac.in

```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.93462632 0.06537368]
```

Result: Successfully trained a decision tree model and used it for prediction.

EXP.NO:08

support vector machine (svm) with difffrent kernels

Aim: To study and implement kernel-based machine learning models.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(X_train,y_train)
pred_linear=model_linear.predict(X_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy:  ",acc_linear)
model_poly=SVC(kernel='poly',degree=4)
model_poly.fit(X_train,y_train)
pred_poly=model_poly.predict(X_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(X_train,y_train)
pred_rbf=model_rbf.predict(X_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
```

output:-

```
print("RBF Kernel Accuracy:",acc_rbf)
```

```
Linear Kernel Accuracy:   1.0
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667
```

Result: Built models using different kernel functions and compared their performance.

EXP.NO:09

Implement K-Nearest Neighbours (KNN) Classification

Aim: To implement the KNN algorithm for classification tasks.

```
[4]
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = pd.read_csv("mobile_price.csv")
X = data.drop("price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Output:-

```
Accuracy: 0.53
Confusion Matrix:
 [[76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]
Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.72      0.71       105
           1       0.34      0.45      0.39        91
           2       0.35      0.37      0.36        92
           3       0.81      0.54      0.65       112

    accuracy                           0.53       400
   macro avg       0.55      0.52      0.53       400
weighted avg       0.57      0.53      0.54       400
```

Result: Successfully classified data using the KNN algorithm and evaluated the model

EXP.NO:10

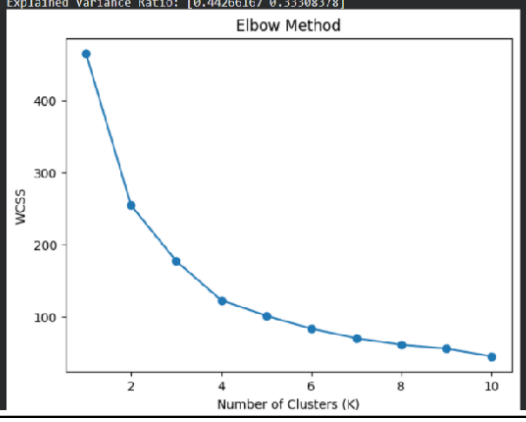
K-Mean Clustering with PCA and Elbow Method

Aim: To perform clustering using K-means and determine the optimal number of clusters using PCA and the elbow method.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("mall coustomer.csv")
X = data[['Age','Annual Income (k$)','Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

Output:-

```
Explained Variance Ratio: [0.44266167 0.33308378]
```



Result: Successfully applied K-means clustering, reduced dimensions using PCA, and identified the best cluster count using the elbow method.